MiniProject 2: Training and Evaluation Practices

Hyperparameter Tuning and Regularization Paths in Logistic Regression

Aahaan Rawal, Ian Gifford, Nathan Hu

Abstract

In this project, we implemented logistic regression for binary classification using mini-batch stochastic gradient descent (SGD) with L2 regularization and applied it to the UCI Spambase dataset. The dataset was intentionally split into a very small training set (5%) and a large held-out test set (95%) in order to emphasize overfitting and highlight the bias-variance trade-off. We first examined optimization behaviour under different batch sizes and learning rates, comparing training dynamics with and without L2 regularization. We then performed K-fold cross validation to tune hyperparameters, including learning rate, batch size, number of epochs, and regularization strength. Next, we explicitly demonstrated the bias-variance trade-off sweeping the L2 regularization parameter and evaluating training and validation performance. Finally, we studied L1-regularized logistic regression to visualize the regularization path, observe sparsity patterns in feature coefficients, and examine the trade-off between interpretability and predictive performance.

Introduction

Email spam detection is a classical binary classification problem that has long served as a benchmark for evaluating machine learning algorithms. In this project, we used the UCI Spambase dataset, which contains 57 continuous features representing word and character frequencies extracted from email messages, along with a binary label indicating whether each message is spam or not. Our primary objective was to implement logistic regression from scratch and train it using mini-batch SGD with L2 regularization. By deliberately restricting the training set to only 5% of the available data (approximately 230 samples), we created a regime in which overfitting is likely, making it possible to clearly observe bias-variance effects and the role of regularization.

The project had four main goals: (1) we aimed to understand the optimization behaviour of SGD under different learning rates and batch sizes. (2) We used K-fold cross-validation to perform principled hyperparameter selection. (3) We analyzed the bias-variance trade-off by sweeping the L2 regularization strength and examining how training and validation errors evolve. (4) Finally, we studied L1-regularized logistic regression to visualize the regularization path and investigate sparsity. Combined, these goals provide a comprehensive view of how optimization choices and regularization strategies affect both model performance and interpretability.

Methods

We implemented binary logistic regression training using mini-batch stochastic gradient descent (SGD). Given the feature matrix $X \in \mathbf{R}^{N \times D}$, where each row is a set of features from a particular instance (D includes a bias term), the labels $y \in \{0, 1\}^N$, and a weight vector $w \in \mathbf{R}^D$, we compute predictions using the logistic function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \text{ where } z = Xw.$$

The objective function we use during training to pick w is given by: $\mathcal{L}(w) = \text{mean CE}(w) + \lambda \|w\|_2^2$, where we wish to minimize the mean cross-entropy loss with L2 regularization, which we control the strength of with λ. It is important to mention that we exclude the bias term from regularization. We minimize the above objective function by using the gradient $\nabla_w(\mathcal{L})$ to update our weights over a set

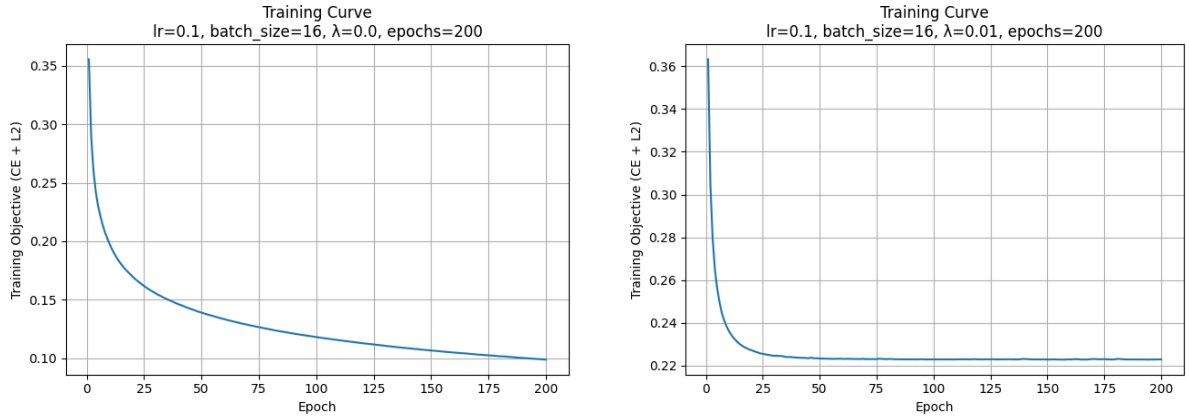
number of epochs. Before the first epoch, we initialize the weights to zero. Then, we begin each epoch by shuffling the training data and partitioning it into mini-batches of an equal set size, called the batch size. For each batch, we compute the gradient only over the given batch and update the weights with the following update rule: $w \leftarrow w - \alpha \nabla_w(\mathcal{L})$, where α is the learning rate.

For preprocessing the data, we noticed that most features were continuous word or character frequency counts that spanned a wide numerical range. Additional features were longest run-length counts of capital letters, which also spanned large ranges. As a result, we standardize each feature to zero mean and unit variance. This ensures that certain features do not dominate gradient updates. This also improves optimization stability, since SGD is more sensitive to feature scale. Comparing mean feature values to maximum feature values also revealed some large outliers, which standardization can help reduce the disproportionate influence of during optimization. Importantly, we computed standardization parameters (mean and standard deviation) only using the training data to prevent data leakage.

Hyperparameters were selected using 5-fold cross validation on the training set. For each fold and hyperparameter combination, the model was trained on *K-1* folds and evaluated on the held-out fold, recording mean and standard deviation of cross-entropy and accuracy. The best hyperparameter configuration was chosen based on the lowest mean cross-entropy loss on validation folds. The final selected hyperparameters were used in subsequent tasks.

Results

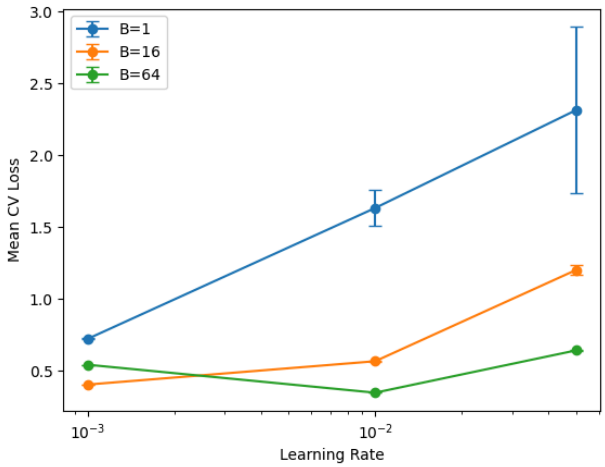
The following training curves plot the training loss across epochs for our initial runs of mini-batch SGD. We observed best results when we set the learning rate to 0.1 and batch size to 16. The left model was trained with no regularization, while the right model was trained with regularization λ=0.01.



Part 2:

mean_loss	std_loss	mean_acc	std_acc	lr	batch_size	epochs	lam
0.349465	0.074679	0.896300	0.041433	0.010	64	100	0.0000
0.359305	0.065695	0.896300	0.041433	0.01	16	200	0.0010
0.387822	0.048980	0.887604	0.034071	0.010	64	50	0.0010

Cross-validation over learning rates $\eta \in \{0.001, 0.01, 0.05\}$, batch sizes $B \in \{1, 16, 64\}$, epochs $\in \{50, 100, 200\}$, and L2 regularization strengths $\lambda \in \{0, 10^{-4}, 10^{-3}\}$ revealed stable performance across λ, with mean accuracies around 89%. While exploratory learning curves suggested $\eta = 0.1$ performed well, cross-validation identified $\eta = 0.010$, a batch size of B = 64, and 100 epochs as yielding the lowest validation cross-entropy. Larger learning rates lead to noticeably higher losses, particularly for smaller batch sizes, reflecting the sensitivity of the optimization process to step size. While the full table provides detailed quantitative results for all tested configurations, we present representative entries illustrating the trends observed, as well as the graph below.



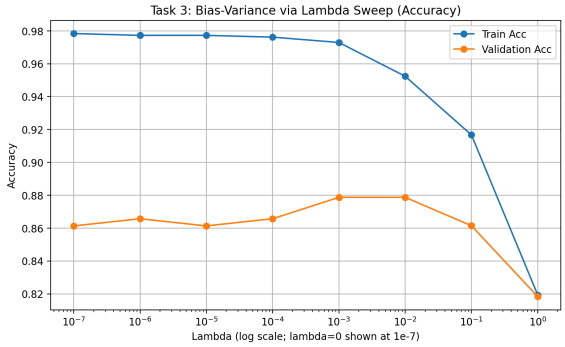
The graph shows a clear interaction between learning rate and batch size by showing mean cross-validation loss (with standard deviation error bars). For $B = 1$, the mean validation loss increases sharply as the learning rate grows, and the error bars widen, indicating high variability and unstable optimization due to noisy gradient estimates. With $B = 16$, the loss curve is flatter but still shows degradation at higher learning rates, suggesting that moderate mini-batches reduce but do not eliminate sensitivity to step size. In contrast, $B = 64$ yields consistently low and stable losses across learning rates, with small error bars that reflect reduced gradient noise and more reliable convergence. Overall, the results show that larger batch size substantially improves optimization stability, while smaller batches require more conservative learning rates to avoid divergence and high variance across folds.

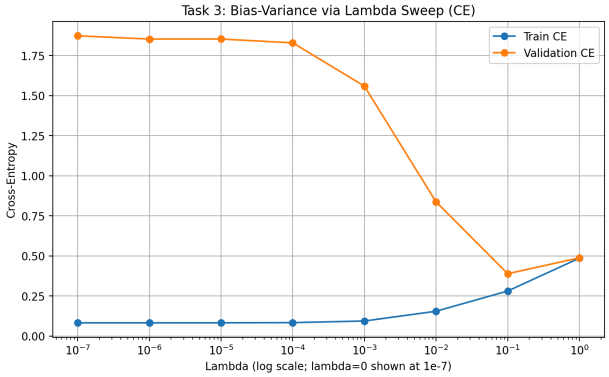
Part 3:

In this task, we investigated the bias-variance trade-off by systematically varying the L_2 regularization parameter, λ, to control model complexity. Regularization introduces a penalty on the magnitude of the model weights, effectively constraining the hypothesis space. By sweeping λ across a logarithmic grid from 0.0 to 1.0, we observed the transition from a high-variance, unregularized model to a high-bias, heavily constrained model.

To ensure robust evaluation and avoid data leakage, we employed 5-fold cross-validation exclusively on our training set. Crucially, feature standardization was performed dynamically within each fold; the mean and standard deviation were computed solely from the training folds and subsequently applied to the validation fold. For this experiment, we held the learning rate (0.1), batch size (16), and epochs (200) constant.

For each λ value, we recorded the mean cross-entropy (CE) loss and classification accuracy across both the training and validation sets. As expected, minimal regularization yielded the lowest training loss but higher validation loss, indicating overfitting (high variance). Conversely, excessive regularization penalized the weights too aggressively, causing both training and validation performance to degrade due to underfitting (high bias). By analyzing the validation cross-entropy curve, we identified the optimal regularization strength to be $\lambda = 0.1$, which minimized our generalization error.

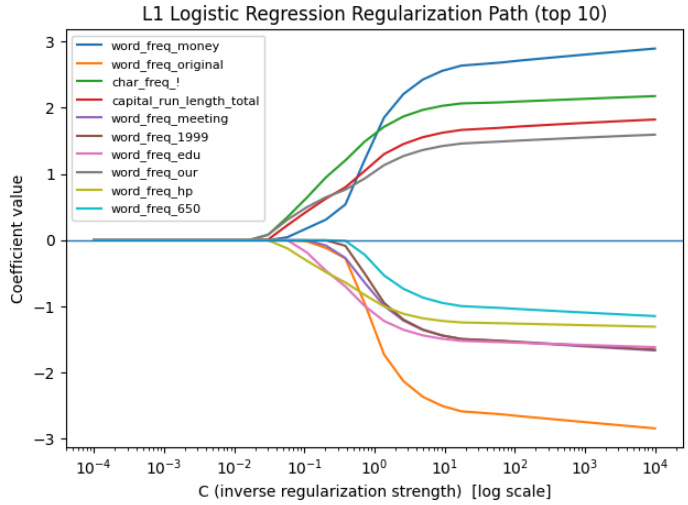




Finally, we retrained our logistic regression model on the entire standardized training set using this optimal λ and evaluated its performance on the held-out test set, providing a final, unbiased estimate of our model's predictive capability. The fully trained model achieved a training cross-entropy loss of 0.293239 with an accuracy of 89.61%. When evaluated on the unseen test data, the model demonstrated strong generalization, yielding a test cross-entropy loss of 0.348216 and a final test accuracy of 88.65%.

Part 4:

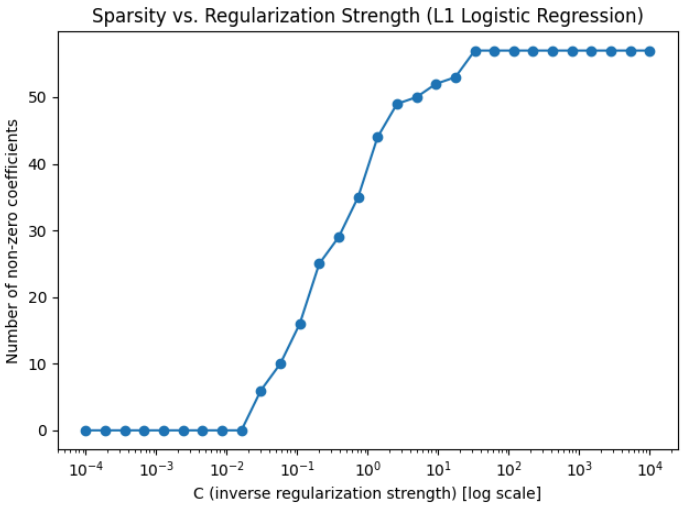
The regularization path shows how model coefficients evolve as L1 regularization varies. In scikit-learn, this is parameterized by $C = 1/\lambda$. Small C corresponds to strong regularization, shrinking many coefficients to zero, while large C corresponds to weak regularization, allowing more coefficients to take non-zero values.



The coefficient-path plot illustrates that when C is very small, all coefficients are zero, producing a highly sparse model. As C increases, the penalty relaxes and the most informative features (such as *word_freq_money, word_freq_original, char_freq_!*) are the first to enter the model with non-zero weights. These early-activating features align with intuitive spam indicators, reflecting their strong discriminative power. With further increases in C, additional features like *word_freq_meeting* and *word_freq_edu* begin to take on non-zero values, and the magnitudes of existing coefficients grow as the model becomes less constrained. By the time C reaches the upper end of the range, the model is substantially less sparse, with larger and more variable coefficients that capture more nuanced patterns but also risk overfitting. Overall, the plot highlights the characteristic sparsity-inducing behaviour of L1 regularization, the order in which features become influential, and the trade-off between interpretability and model complexity as regularization strength varies.

The sparsity plot shows how the L1-regularized logistic regression model transitions from an extremely simple, highly constrained classifier to a much richer one as the regularization weakens. At very small

values of C, nearly all coefficients are forced to be zero, producing a model that uses almost no features. As C increases, the regularization relaxes and the model begins to admit more non-zero coefficients. The curve rises sharply once C passes roughly 10^{-2}, indicating that the model suddenly finds it worthwhile to include a substantial number of features once the penalty becomes moderate rather than severe. Beyond this point, the number of active coefficients continues to grow until it plateaus at around 55, reflecting the point at which nearly all informative features have entered the model and further increases in C no longer meaningfully change sparsity.



Features that become non-zero at small C are considered most important because they are retained even when the model is highly constrained. Conversely, features that only appear at larger C values are less informative. L1 regularization provides a balance between sparsity and predictive accuracy. Strong regularization (high sparsity) yields a simple, interpretable model that highlights only the most important features but may underfit the training data. Weak regularization (low sparsity) allows the model to leverage many features, often improving predictive performance but at the cost of interpretability.

Discussion and Conclusion

Our initial training curves of mini-batch SGD show that the models with the chosen hyperparameters converged reliably with training loss decreasing over epochs. Without regularization, the loss decreased faster and reached a lower final value. With L2 regularization, the loss plateaus much earlier, and the final loss is higher due to the penalty term. This is a clear sign of weight decay, and shows how regularization limits how large the model parameters can get. The cross-validation results further emphasized the importance of principled hyperparameter tuning. While several configurations achieved similar accuracies, validation cross-entropy revealed meaningful differences in generalization performance. Selecting hyperparameters based on validation loss ensured that the final model was not simply fitting noise in the small training split, but instead achieved stable performance across folds.

The λ-sweep provided a clear illustration of the bias-variance trade-off: very small regularization strengths produced low training loss but substantially higher validation loss, demonstrating high variance and overfitting. As λ increased, the gap between training and validation performance narrowed. Excessively large λ values, however, degraded both training and validation performance, indicating underfitting due to high bias. This progression confirms the theoretical expectation that optimal generalization occurs at a balance between model flexibility and constraint. L1 regularization reveals feature sparsity patterns: strong regularization yields highly sparse, interpretable models, while weaker regularization allows more features to contribute, increasing expressiveness at the cost of simplicity. Features that remain active under strong penalties can be interpreted as the most informative predictors of spam, whereas features that only appear at weak regularization levels likely provide marginal or redundant information. Overall, the project demonstrates how careful choices in optimization, regularization, and hyperparameter selection influence both predictive accuracy and interpretability in logistic regression.

Statement of Contributions

Ian handled Task 1+4, Aahaan Task 2 and Nathan Task 3. We completed our tasks separately on the notebook and writeup, although Aahaan covered Ian's writeup parts since he handled two programming tasks. We collaboratively proofread the writeup and made sure our results made sense together.

Statement on the use of LLMs and other resources

LLMs were used to help us justify certain decisions, and to assist development whenever we were stuck and had invested significant time with little success. This looked like assistance in planning development by providing feedback and clarification on how to appropriately apply course concepts, and were not used in implementing solutions or writing code.